

27-04-2026

Producer consumer problem

Code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#include <unistd.h>
```

```
int buffer_size;
```

```
int num_items;
```

```
int *buffer;
```

```
int in = 0;
```

```
int out = 0;
```

```
sem_t empty;
```

```
sem_t full;
```

```
pthread_mutex_t mutex;
```

```
void* producer(void* param) {
```

```
    int producer_id = *((int*)param);
```

```
    for (int i = 0; i < num_items; i++) {
```

```
        int item = rand() % 100;
```

```
        sem_wait(&empty);
```

```
        pthread_mutex_lock(&mutex);
```

```
        buffer[in] = item;
```

```
        printf("Producer %d: Inserted item %d at index %d\n", producer_id, item, in);
```

```
        in = (in + 1) % buffer_size;
```

```
        pthread_mutex_unlock(&mutex);
```

```

    sem_post(&full);

    sleep(1);
}
return NULL;
}

void* consumer(void* param) {
    int consumer_id = *((int*)param);

    for (int i = 0; i < num_items; i++) {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumer %d: Removed item %d from index %d\n", consumer_id, item, out);
        out = (out + 1) % buffer_size;
        pthread_mutex_unlock(&mutex);

        sem_post(&empty);

        sleep(2);
    }
    return NULL;
}

int main() {
    int num_producers, num_consumers;

    printf("Enter Buffer Size: ");
    scanf("%d", &buffer_size);
    printf("Enter Number of Producers: ");

```

```

scanf("%d", &num_producers);

printf("Enter Number of Consumers: ");

scanf("%d", &num_consumers);

printf("Enter Number of Items per thread to produce/consume: ");

scanf("%d", &num_items);

buffer = (int*)malloc(buffer_size * sizeof(int));

pthread_t *producers = (pthread_t*)malloc(num_producers * sizeof(pthread_t));

pthread_t *consumers = (pthread_t*)malloc(num_consumers * sizeof(pthread_t));

int *prod_ids = (int*)malloc(num_producers * sizeof(int));

int *cons_ids = (int*)malloc(num_consumers * sizeof(int));


pthread_mutex_init(&mutex, NULL);

sem_init(&empty, 0, buffer_size);

sem_init(&full, 0, 0);


printf("\n--- Starting Simulation ---\n");

for (int i = 0; i < num_producers; i++) {

    prod_ids[i] = i + 1;

    pthread_create(&producers[i], NULL, producer, &prod_ids[i]);

}

for (int i = 0; i < num_consumers; i++) {

    cons_ids[i] = i + 1;

    pthread_create(&consumers[i], NULL, consumer, &cons_ids[i]);

}


for (int i = 0; i < num_producers; i++) {

    pthread_join(producers[i], NULL);

}

for (int i = 0; i < num_consumers; i++) {

    pthread_join(consumers[i], NULL);

}


pthread_mutex_destroy(&mutex);

sem_destroy(&empty);

```

```

sem_destroy(&full);

free(buffer);
free(producers);
free(consumers);
free(prod_ids);
free(cons_ids);

printf("--- All threads have finished execution ---\n");

return 0;
}

```

output:

```

Enter Buffer Size: 5
Enter Number of Producers: 2
Enter Number of Consumers: 2
Enter Number of Items per thread to produce/consume: 5

--- Starting Simulation ---
Producer 1: Inserted item 41 at index 0
Producer 2: Inserted item 41 at index 1
Consumer 1: Removed item 41 from index 0
Consumer 2: Removed item 41 from index 1
Producer 1: Inserted item 67 at index 2
Producer 2: Inserted item 67 at index 3
Producer 2: Inserted item 34 at index 4
Producer 1: Inserted item 34 at index 0
Consumer 2: Removed item 67 from index 2
Consumer 1: Removed item 67 from index 3
Producer 1: Inserted item 0 at index 1
Producer 2: Inserted item 0 at index 2
Consumer 1: Removed item 34 from index 4
Consumer 2: Removed item 34 from index 0
Producer 2: Inserted item 69 at index 3
Producer 1: Inserted item 69 at index 4
Consumer 1: Removed item 0 from index 1
Consumer 2: Removed item 0 from index 2
Consumer 2: Removed item 69 from index 3
Consumer 1: Removed item 69 from index 4
--- All threads have finished execution ---
PS D:\vishwanath\OS LAB> 

```